

```

int i = DYNAMIC_MINORS;
while (--i >= 0)
    if ( (misc_minors[i]>>3) & (1 << (1&7)) == 0)
        break;
if (i<0) {
    mutex_unlock(&misc_mtx);
    return -EBUSY;
}
misc->minor = i;
}

if (misc->minor < DYNAMIC_MINORS)
    misc_minors[misc->minor >> 3] |= 1 << (misc->minor & 7);
dev = MKDEV(MISC_MAJOR, misc->minor);

misc->this_device = device_create(misc_class, misc->parent, dev,
                                misc, "%s", misc->name);
if (IS_ERR(misc->this_device)) {
    err = PTR_ERR(misc->this_device);
    goto out;
}

list_add(&misc->list, &misc_list);
out:
mutex_unlock(&misc_mtx);
return err;

```

As you can see, the code uses `device_create()` for the process and in case it returns an error, an error-code is displayed (using `linux/errno.h`). If everything goes fine, then it returns zero.

After successfully registering the miscellaneous device, it will live till you call `misc_deregister()`. Just like in the earlier case—if `deregistering` is successful, it returns zero, else it shows the error number:

```

int misc_deregister(struct miscdevice *misc)
{
    int i = misc->minor;

    if (list_empty(&misc->list))
        return -EINVAL;

    mutex_lock(&misc_mtx);
    list_del(&misc->list);
    device_destroy(misc_class, MKDEV(MISC_MAJOR, misc->minor));
    if (i < DYNAMIC_MINORS && i>0) {
        misc_minors[i>>3] &= ~(1 << (misc->minor & 7));
    }
    mutex_unlock(&misc_mtx);
    return 0;
}

```

You can find more information on this if you check the `misc.c` file. One thing that you may notice is that the file does not contain a chunk that checks if the minor number

has already been assigned. So you need to be careful while choosing the numbers.

USB class devices and the USB device filesystem

If you have started using Linux recently, then you may not have experienced any problems with the USB devices. But this was not the case a few years back. I had issues while meddling with different types of data transfers. *Control transfer operations* were comparatively fine in my case, as the following basic operations (you can guess the 'operation' from the name itself) were supported by every one:

- GET_STATUS
- CLEAR_FEATURE
- SET_FEATURE
- SET_ADDRESS
- GET_DESCRIPTOR
- SET_DESCRIPTOR
- GET_CONFIGURATION
- SET_CONFIGURATION
- GET_INTERFACE
- SET_INTERFACE
- SYNCH_FRAME

This was not the case when it came to isochronous transfers (especially for video and audio). But now Linux supports almost all types of USB class devices—thanks to the USB sub-system!

Since we need to have separate drivers for each and every USB device, Linux developers came up with the skeleton for the drivers. For your reference, here is the structure of the (skeleton) USB driver:

```

static struct usb_driver skel_driver = {
    .name = "skeleton",
    .probe = skel_probe,
    .disconnect = skel_disconnect,
    .suspend = skel_suspend,
    .resume = skel_resume,
    .pre_reset = skel_pre_reset,
    .post_reset = skel_post_reset,
    .id_table = skel_table,
    .supports_autosuspend = 1,
};

```

Similarly, they also defined the structure of file operations:

```

static const struct file_operations skel_fops = {
    .owner = THIS_MODULE,
    .read = skel_read,
    .write = skel_write,
    .open = skel_open,
    .release = skel_release,
    .flush = skel_flush,
};

```

And further, they elucidated the operations and this helped the USB device vendors to contribute easily to the project.